

17 State pattern

State pattern

Intent

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Advantage

- Implementation of polymorphic behavior is clearly visible.
- The chances of error are less and it's very easy to add more states for additional behavior. Thus making our code more robust, easily maintainable and flexible.
- Also State pattern helped in avoiding if-else or switch-case conditional logic in this scenario.

>> สร้างคลาสแทน state ต่าง ๆ เพื่อใช้งาน แทนที่จะมี variable state เดียว แล้ว check IF state

Reference

[1] State Design Pattern Example

<https://www.javacodegeeks.com/2015/09/state-design-pattern.html>

ตัวอย่างที่ 1 Robot

```
1. public interface RoboticState {
2.     public void walk();
3.     public void cook();
4.     public void off();
5. }

1. public class RoboticOn implements RoboticState {
2.
3.     private final Robot robot;
4.
5.     public RoboticOn(Robot robot) {
6.         this.robot = robot;
7.     }
8.     @Override
9.     public void walk() {
10.         System.out.println("Walking...");
11.     }
12.     @Override
13.     public void cook() {
14.         System.out.println("Cooking...");
15.         robot.setRoboticState(robot.getRoboticCook());
16.     }
17.     @Override
18.     public void off() {
19.         robot.setState(robot.getRoboticOff());
20.         System.out.println("Robot is switched off");
21.     }
22. }

1. public class RoboticCook implements RoboticState {
2.
3.     private final Robot robot;
4.
5.     public RoboticCook(Robot robot) {
6.         this.robot = robot;
7.     }
8.     @Override
9.     public void walk() {
10.         System.out.println("Walking...");
11.         robot.setRoboticState(robot.getRoboticOn());
12.     }
13.     @Override
14.     public void cook() {
15.         System.out.println("Cooking...");
16.     }
17.     @Override
18.     public void off() {
19.         System.out.println("Cannot switched off while cooking...");
20.     }
21. }

1. public class Robot implements RoboticState {
2.
3.     private RoboticState roboticOn;
4.     private RoboticState roboticCook;
5.     private RoboticState roboticOff;
6.
7.     private RoboticState state;
8.
9.     public Robot() {
10.         this.roboticOn = new RoboticOn(this);
11.         this.roboticCook = new RoboticCook(this);
12.         this.roboticOff = new RoboticOff(this);
13.
14.         this.state = roboticOn;
15.     }
```

```

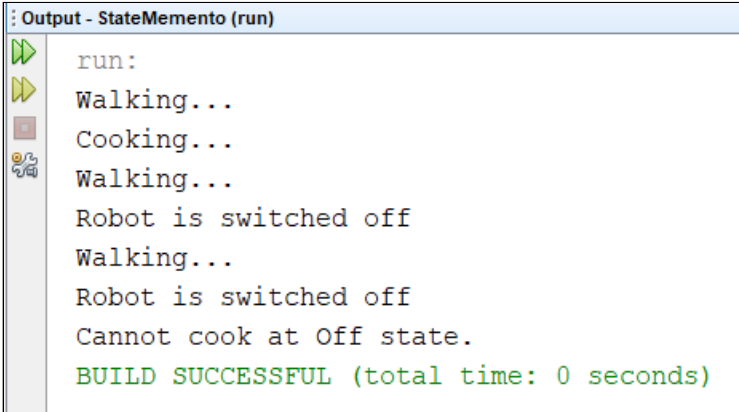
16.
17.     public void setRoboticState(RoboticState state) {
18.         this.state = state;
19.     }
20.     @Override
21.     public void walk() {
22.         state.walk();
23.     }
24.     @Override
25.     public void cook() {
26.         state.cook();
27.     }
28.     @Override
29.     public void off() {
30.         state.off();
31.     }
32.
33.     public RoboticState getRoboticOn() {
34.         return roboticOn;
35.     }
36.     public void setRoboticOn(RoboticState roboticOn) {
37.         this.roboticOn = roboticOn;
38.     }
39.     public RoboticState getRoboticCook() {
40.         return roboticCook;
41.     }
42.     public void setRoboticCook(RoboticState roboticCook) {
43.         this.roboticCook = roboticCook;
44.     }
45.     public RoboticState getRoboticOff() {
46.         return roboticOff;
47.     }
48.     public void setRoboticOff(RoboticState roboticOff) {
49.         this.roboticOff = roboticOff;
50.     }
51.     public RoboticState getState() {
52.         return state;
53.     }
54.     public void setState(RoboticState state) {
55.         this.state = state;
56.     }
57.
58. }

```

```

1. public class DemoRobot {
2.
3.     public static void main(String[] args) {
4.         Robot robot = new Robot();
5.         robot.walk();
6.         robot.cook();
7.         robot.walk();
8.         robot.off();
9.
10.        robot.walk();
11.        robot.off();
12.        robot.cook();
13.
14.    }
15. }

```



```

Output - StateMemento (run)
run:
Walking...
Cooking...
Walking...
Robot is switched off
Walking...
Robot is switched off
Cannot cook at Off state.
BUILD SUCCESSFUL (total time: 0 seconds)

```

ตัวอย่างที่ 2 TV

```
1. public interface State {
2.     public void turnOn();
3.     public void turnOff();
4. }

1. public class TV implements State {
2.
3.     private State tvOn;
4.     private State tvOff;
5.     private State state;
6.
7.     public TV() {
8.         this.tvOn = new TVOnState(this);
9.         this.tvOff = new TVOffState(this);
10.        this.state = tvOn;
11.    }
12.
13.    @Override
14.    public void turnOn() {
15.        state.turnOn();
16.    }
17.
18.    @Override
19.    public void turnOff() {
20.        state.turnOff();
21.    }
22.
23.    public State getTvOn() {
24.        return tvOn;
25.    }
26.
27.    public void setTvOn(State tvOn) {
28.        this.tvOn = tvOn;
29.    }
30.
31.    public State getTvOff() {
32.        return tvOff;
33.    }
34.
35.    public void setTvOff(State tvOff) {
36.        this.tvOff = tvOff;
37.    }
38.
39.    public State getState() {
40.        return state;
41.    }
42.
43.    public void setState(State state) {
44.        this.state = state;
45.    }
46.
47. }
```

```

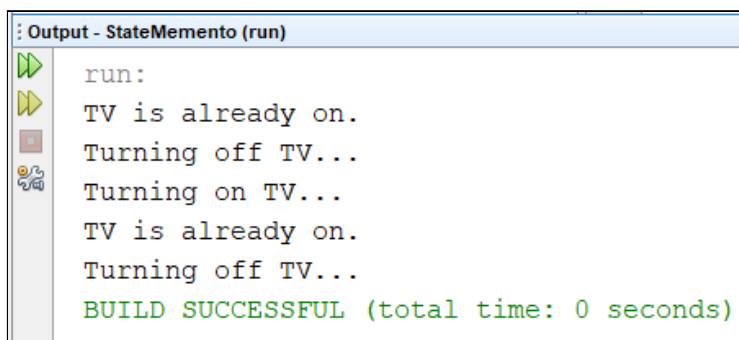
1. public class TVOnState implements State {
2.
3.     private final TV tv;
4.
5.     public TVOnState(TV tv) {
6.         this.tv = tv;
7.     }
8.
9.     @Override
10.    public void turnOn() {
11.        System.out.println("TV is already on.");
12.    }
13.    @Override
14.    public void turnOff() {
15.        tv.setState(tv.getTvOff());
16.        System.out.println("Turning off TV...");
17.    }
18.
19. }

```

```

1. public class DemoTV {
2.
3.     public static void main(String[] args) {
4.         TV tv = new TV();
5.         tv.turnOn();
6.         tv.turnOff();
7.         tv.turnOn();
8.         tv.turnOn();
9.         tv.turnOff();
10.    }
11. }

```



```

Output - StateMemento (run)
run:
TV is already on.
Turning off TV...
Turning on TV...
TV is already on.
Turning off TV...
BUILD SUCCESSFUL (total time: 0 seconds)

```